

CS 473 (Spring 2025)

Homework 4 Solutions

Problem 4.1: Suppose we have a Las Vegas randomized algorithm A that runs in μ expected number of steps. As mentioned in class, for any given $t > 1$, we can obtain a new Monte Carlo algorithm A' that runs within $t\mu$ steps by outputting “garbage” if the time limit is exceeded. The error probability is at most $1/t$ by Markov’s inequality.

But we can do better! For example, we can run A for up to 2μ steps, and if it fails, rerun A for another 2μ steps (with an independent sequence of random bits), etc., for $\lfloor t/2 \rfloor$ rounds. This gives a Monte Carlo algorithm with the total number of steps still bounded by $t\mu$, and the error probability reduced to $1/2^{\lfloor t/2 \rfloor} = O(1/2^{t/2}) \leq O(0.708^t)$, which decays exponentially!

In this question, you will analyze further improvements to the base of the exponential:

- (a) (25 pts) Let $X \geq 0$ be a random variable with $\mathbb{E}[X] = \mu$. Given $\alpha_1, \alpha_2, \dots, \alpha_k \geq 0$, let $p_1 = \Pr[X \geq \alpha_1\mu]$, $p_2 = \Pr[X \geq (\alpha_1 + \alpha_2)\mu]$, $\dots$, $p_k = \Pr[X \geq (\alpha_1 + \dots + \alpha_k)\mu]$.

Prove the following inequality:

$$\alpha_1 p_1 + \alpha_2 p_2 + \dots + \alpha_k p_k \leq 1.$$

[Hint: modify the proof of Markov’s inequality and use telescoping sum...]

- (b) (25 pts) Now, consider a strategy with three iterations where we run A for $\alpha_1\mu$ steps in the first iteration, $(\alpha_1 + \alpha_2)\mu$ steps in the second iteration, and $(\alpha_1 + \alpha_2 + \alpha_3)\mu$ steps in the third iteration.

With this strategy, we obtain a Monte Carlo algorithm with the total number of steps upper-bounded by $(3\alpha_1 + 2\alpha_2 + \alpha_3)\mu$. Bound the error probability purely as a function of α_1 , α_2 , and α_3 .

[Hint: you may use the fact that $x + y + z \leq 1$ ($x, y, z \geq 0$) implies $xyz \leq 1/27$. (This follows from the standard inequality that the “geometric mean” is at most the “arithmetic mean”.)]

- (c) (25 pts) Using part (b), show how to obtain a Monte Carlo algorithm with the total number of steps bounded by $t\mu$, and the error probability at most $O(0.545^t)$ (thus, improving $O(0.708^t)$).

[Hint: repeat for $\left\lfloor \frac{t}{3\alpha_1 + 2\alpha_2 + \alpha_3} \right\rfloor$ rounds; set $\alpha_1 = c/3$, $\alpha_2 = c/2$, $\alpha_3 = c$ for some choice of c (you might need a calculator)...]

- (d) (25 pts) Generalize part (b) from 3 iterations to k iterations for a larger constant k , and show how to reduce the error probability further to below $O(0.4^t)$.

[Hint: choose some k below 50, and choose $c = 1 \dots$]

Solution:

$$\begin{aligned}
(a) \quad \mu = \mathbb{E}[X] &= \sum_x x \cdot \Pr[X = x] \\
&\geq \sum_{\alpha_1 \mu \leq x < (\alpha_1 + \alpha_2) \mu} \alpha_1 \mu \cdot \Pr[X = x] + \\
&\quad \sum_{(\alpha_1 + \alpha_2) \mu \leq x < (\alpha_1 + \alpha_2 + \alpha_3) \mu} (\alpha_1 + \alpha_2) \mu \cdot \Pr[X = x] + \cdots + \\
&\quad \sum_{(\alpha_1 + \cdots + \alpha_{k-1}) \mu \leq x < (\alpha_1 + \cdots + \alpha_k) \mu} (\alpha_1 + \cdots + \alpha_{k-1}) \mu \cdot \Pr[X = x] + \\
&\quad \sum_{x \geq (\alpha_1 + \cdots + \alpha_k) \mu} (\alpha_1 + \cdots + \alpha_k) \mu \cdot \Pr[X = x] \\
&= \alpha_1 \mu \cdot (p_1 - p_2) + \\
&\quad (\alpha_1 + \alpha_2) \mu \cdot (p_2 - p_3) + \cdots + \\
&\quad (\alpha_1 + \cdots + \alpha_{k-1}) \mu \cdot (p_{k-1} - p_k) + \\
&\quad (\alpha_1 + \cdots + \alpha_k) \mu \cdot p_k \\
&= \alpha_1 \mu p_1 + \alpha_2 \mu p_2 + \cdots + \alpha_k \mu p_k.
\end{aligned}$$

Dividing μ on both sides, we obtain $1 \geq \alpha_1 p_1 + \alpha_2 p_2 + \cdots + \alpha_k p_k$.

- (b) The error probability is $p_1 p_2 p_3$, where p_1, p_2, p_3 are as defined in (a) with X being the random variable corresponding to the number of steps required by algorithm A.

Since $\alpha_1 p_1 + \alpha_2 p_2 + \alpha_3 p_3 \leq 1$, we know that $\alpha_1 p_1 \alpha_2 p_2 \alpha_3 p_3 \leq 1/27$. Thus, $p_1 p_2 p_3 \leq \frac{1}{27 \alpha_1 \alpha_2 \alpha_3}$. We conclude that the error probability is at most $\frac{1}{27 \alpha_1 \alpha_2 \alpha_3}$.

- (c) We repeat for $m = \left\lfloor \frac{t}{3\alpha_1 + 2\alpha_2 + \alpha_3} \right\rfloor$ rounds. Since each round takes at most $(3\alpha_1 + 2\alpha_2 + \alpha_3)\mu$ steps, the total number of steps is at most $m \cdot (3\alpha_1 + 2\alpha_2 + \alpha_3)\mu \leq t\mu$. We output “garbage” when none of the runs terminates within the stated number of steps.

By independence of the rounds, the error probability is at most

$$\left(\frac{1}{27 \alpha_1 \alpha_2 \alpha_3} \right)^m \leq O \left(\left(\frac{1}{27 \alpha_1 \alpha_2 \alpha_3} \right)^{t/(3\alpha_1 + 2\alpha_2 + \alpha_3)} \right).$$

As suggested by the hint, we set $\alpha_1 = c/3$, $\alpha_2 = c/2$, and $\alpha_3 = c$ for some c . Then the above expression simplifies to

$$O \left(\left[\left(\frac{6}{27c^3} \right)^{1/(3c)} \right]^t \right).$$

Choosing $c = 1.65$, we get $O(0.545^t)$.

[*Remark:* the choice for c can be seen by plotting the function $f(c) = \left(\frac{6}{27c^3} \right)^{1/(3c)}$. Alternatively, we can use calculus to minimize f . (E.g., define $g(x) = \ln f(1/x) =$

$\frac{1}{3}x \ln(\frac{2}{9}x^3) = \frac{1}{3}x(\ln \frac{2}{9} + 3 \ln x)$, and solve the equation $g'(x) = \frac{1}{3}(\ln \frac{2}{9} + 3 \ln x + 3) = 0$, giving $\ln x = \frac{1}{3} \ln \frac{9}{2} - 1$, i.e., $x = (\frac{9}{2})^{1/3}e^{-1}$. Thus, the optimal choice for c is $c = (\frac{2}{9})^{1/3}e$.]

- (d) Each round has k iterations: we run A for $\alpha_1\mu$ steps in the first iteration, $(\alpha_1 + \alpha_2)\mu$ steps in the second iteration, $\dots$, and $(\alpha_1 + \dots + \alpha_k)\mu$ steps in the k -th iteration. Each round takes at most $(k\alpha_1 + (k-1)\alpha_2 + \dots + \alpha_k)\mu$ steps. We repeat for $m = \left\lfloor \frac{t}{k\alpha_1 + (k-1)\alpha_2 + \dots + \alpha_k} \right\rfloor$ rounds. We output “garbage” when none of the runs terminates within the stated number of steps. The total number of steps is at most $t\mu$.

The error probability of one round is $p_1 p_2 \dots p_k$, where $p_1, \dots, p_k$ are as defined in (a) with X being the random variable corresponding to the number of steps required by algorithm A. Since $\alpha_1 p_1 + \dots + \alpha_k p_k \leq 1$, we know that $\alpha_1 p_1 \dots \alpha_k p_k \leq (1/k)^k$ (by the arithmetic/geometric mean inequality). Thus, the error probability of one round is at most $p_1 \dots p_k \leq \frac{1}{k^k \alpha_1 \dots \alpha_k}$.

The overall error probability is at most

$$\left(\frac{1}{k^k \alpha_1 \dots \alpha_k} \right)^m \leq O \left(\left(\frac{1}{k^k \alpha_1 \dots \alpha_k} \right)^{t/(k\alpha_1 + (k-1)\alpha_2 + \dots + \alpha_k)} \right).$$

Setting $\alpha_1 = 1/k$, $\alpha_2 = 1/(k-1)$, $\dots$, $\alpha_k = 1$, the probability is at most

$$O \left(\left[\left(\frac{k!}{k^k} \right)^{1/k} \right]^t \right).$$

Plugging in $k = 35$, this gives at most $O(0.398^t)$.

[*Remark:* By Stirling’s formula, $k! \approx \sqrt{2\pi k}(k/e)^k$, implying that $(\frac{k!}{k^k})^{1/k}$ converges to $1/e < 0.3679$.]

[*Remark:* this problem is based on a paper by Alt, Guibas, Mehlhorn, Karp, and Wigderson (1996), who obtained the $1/e$ bound (Theorem 3) as well as other results in related settings.]

Problem 4.2: In a popular form of logic puzzles, you land on an island that has three types of inhabitants: “knights”, who always tell the truth; “knaves”, who always lie; and “spies”, who sometimes lie and sometimes tell the truth.

Suppose there are n inhabitants, where 60% are known to be decent folks, i.e., knights. The remaining 40% are bad, i.e., knaves or spies. You want to know who are the good/bad guys, i.e., determine the types of all n inhabitants. You are allowed to ask only questions of the form, “is person A a knight/knave/spy?”, to another person B. (All the inhabitants know each other.) Obviously, if you can find a person who you know is a knight, the problem can be solved after asking n additional questions.

- (a) (10 pts) Give a (very) efficient Monte Carlo algorithm that finds a knight. State the probability of error. (Hint: this is supposed to be very easy!)

- (b) (90 pts) Give a Las Vegas algorithm that finds a knight by asking $O(n)$ expected number of questions. Analyze the constant factor in the big-Oh and make it smaller than 1.5.
 [Hint: use (a). How can you confirm whether a specific person is a knight by asking $O(n)$ questions?]
 [Note: there is also a deterministic algorithm that requires $O(n)$ questions, but it is more complicated and has a larger constant factor.]

Solution:

- (a) Just pick a person at random! No questions need to be asked. The probability of error is 0.4.
 (b) *Pseudocode:*

```

0. repeat {
1.   pick a person A at random
2.   for each of  $0.8n + 1$  persons B do
3.     ask B whether A is a knight
4.     if at least of these  $0.4n + 1$  persons B replies "yes" then return A
}
```

Note that if A is indeed a knight, then the test in line 4 must hold, since at least $0.4n + 1$ of the persons asked are knights (since the number of knaves/spies is at most $0.4n$). On the other hand, if A is not a knight, then the test would fail, again since at least $0.4n + 1$ of the persons asked are knights. So the algorithm always correctly returns a knight.

Analysis: Let E_t be the event that A is not a knight at the t -th iteration. We know that $\Pr(E_t) = 0.4$.

The number of questions asked at the t -th iteration is $0.8n + 1$. The t -th iteration exists only if $E_1, \dots, E_{t-1}$ all holds. Thus, the expected number of questions asked is

$$\begin{aligned}
 &\leq \sum_{t=1}^{\infty} \Pr(E_1 \cap \dots \cap E_{t-1})(0.8n + 1) = \sum_{t=1}^{\infty} 0.4^{t-1}(0.8n + 1) \\
 &= \frac{1}{1 - 0.4}(0.8n + 1) = 1.333 \dots n + O(1).
 \end{aligned}$$

[Note: alternatively, one can invoke properties of exponentially distributed random variables to infer directly that the expected number of iterations is $1/0.6$.]

Problem 4.3: We are given a set S of strings of total length N over the binary alphabet $\{0, 1\}$. We are also given a number ℓ . We want to find a palindrome p of length ℓ and two distinct strings $s, s' \in S$ such that their concatenation ss' contains p as a substring.

For example, for $S = \{0111, 1111010, 10110, 11011\}$ and $\ell = 7$, the concatenation of 1111010 and 11011 contains the palindrome 1101011.

Describe a randomized Las Vegas algorithm for this problem, by using the Karp–Rabin fingerprint technique (to be covered in Feb 28’s lecture). The expected running time should be $O(N \log N)$ or better.

[Hint: for every string $s \in S$, pre-compute fingerprints of each prefix/suffix of s and the reverse of s . You may find subtraction useful...]

Solution: Let $S = \{s_1, \dots, s_n\}$ where $\sum_{i=1}^n |s_i| = N$. Let $s_i = a_{i,1} \dots a_{i,|s_i|}$. Let $u_{i,k}$ denote the length- k suffix of s_i , and $v_{j,k}$ denote the length- $(\ell - k)$ prefix of s_j . We want to find $i, j \in \{1, \dots, n\}$ ($i \neq j$) and $k \in \{0, \dots, \ell\}$ such that $u_{i,k}v_{j,k}$ is a palindrome, i.e., $u_{i,k}v_{j,k} = v_{j,k}^R u_{i,k}^R$, where v^R denotes the reverse of a string v .

The idea is to find i, j, k such that $F_x(u_{i,k}v_{j,k}) = F_x(v_{j,k}^R u_{i,k}^R)$, where F_x is the fingerprint function from class: $F_x(a_{m-1} \dots a_0) := (a_{m-1}x^{m-1} + \dots + a_1x + a_0) \bmod p$, for a sufficiently large prime number p . Let $f_{i,k} = F_x(u_{i,k})$, $f'_{i,k} = F_x(u_{i,k}^R)$, $g_{j,k} = F_x(v_{j,k})$, $g'_{j,k} = F_x(v_{j,k}^R)$. This condition is equivalent to $f_{i,k}x^{\ell-k} + g_{j,k} \equiv g'_{j,k}x^k + f'_{i,k} \pmod{p}$, i.e.,

$$f_{i,k}x^{\ell-k} - f'_{i,k} \equiv g'_{j,k}x^k - g_{j,k} \pmod{p}.$$

We can precompute the fingerprints for all the suffixes and prefixes by a variant of the “rolling” trick from the Rabin–Karp algorithm.

A remaining case¹ is when a palindrome of length ℓ is contained entirely in one string of S . This case is a little easier to handle. Let $w_{i,k}$ denote the length- ℓ prefix of $u_{i,k}$ (assuming $k \geq \ell$). We want to find i and k such that $w_{i,k} = w_{i,k}^R$. We do so by finding i, k with $F_x(w_{i,k}) = F_x(w_{i,k}^R)$. Note that $F_x(w_{i,k})x^{k-\ell} \equiv (f_{i,k} - f_{i,k-\ell}) \pmod{p}$ and $F_x(w_{i,k}^R)x^{k-\ell} \equiv (f'_{i,k} - f'_{i,k-\ell}) \pmod{p}$.

Pseudocode.

¹We will not take off points if you forgot this case.

0. $x = \text{random}(0, p - 1)$
1. for $i = 1$ to n do
2. $f_{i,0} = f'_{i,0} = 0$
3. for $k = 1$ to $|s_i|$ do
4. $f_{i,k} = (f_{i,k-1} + a_{i,|s_i|-k+1}x^{k-1}) \bmod p$
5. $f'_{i,k} = (f'_{i,k-1}x + a_{i,|s_i|-k+1}) \bmod p$
6. $f''_{i,k} = (f_{i,k}x^{\ell-k} - f'_{i,k}) \bmod p$
7. for $j = 1$ to n do
8. $g_{i,\ell} = g'_{i,\ell} = 0$
9. for $k = \ell - 1$ down to $\ell - |s_i|$ do
10. $g_{j,k} = (g_{j,k+1}x + a_{j,\ell-k}) \bmod p$
11. $g'_{j,k} = (g'_{j,k+1} + a_{j,\ell-k}x^{\ell-k-1}) \bmod p$
12. $g''_{j,k} = (g'_{j,k}x^k - g_{j,k}) \bmod p$
13. for $k = 0$ to ℓ do
14. $F''_k = \{f''_{1,k}, \dots, f''_{n,k}\}, G''_k = \{g''_{1,k}, \dots, g''_{n,k}\}$
15. if F''_k and G''_k contain a common element $f''_{i,k} = f''_{j,k}$ ($i \neq j$) then
16. if $u_{i,k}v_{j,k}$ is a palindrome then return yes
17. else return answer by brute force
18. for $i = 1$ to n do
19. for $k = \ell$ to $|s_i|$ do
20. if $f_{i,k} - f_{i,k-\ell} \equiv f'_{i,k} - f'_{i,k-\ell} \pmod{p}$ then
21. if $w_{i,k}$ is a palindrome then return yes
22. else return answer by brute force
23. return no

The algorithm is always correct (Las Vegas): if no i, j exist in line 15 for all k , then the answer is no. If the test in line 16 succeeds, we have verified that the answer is yes. Otherwise, we run brute force in line 17.

Analysis. We can first precompute $1, x, x^2, \dots, x^\ell$ with $\ell - 1$ multiplications. Afterwards, lines 4–6 and 10–12 takes $O(1)$ arithmetic operations. So, the total number of arithmetic operations is $O(\sum_i |s_i|) = O(N)$.

Line 15 requires further details, since trivially it would take quadratic time. One way is to sort $F''_k \cup G''_k$ in $O(n \log n)$ time, for example, by heapsort. Then we can divide the sorted list into buckets, where each bucket holds elements with identical values. For each bucket, we can see if it contains two elements $f''_{i,k}$ and $g''_{j,k}$ with $i \neq j$. If so, we have found the desired common element. Thus, line 15 can be done in $O(n \log n)$ time.

The test in line 16 takes $O(\ell) \leq O(N)$ time.

For line 17 or 22, a brute-force algorithm takes $O(n^2\ell) \leq O(N^3)$ time naively, but we argue that the probability that this step is performed is small: Let E_{ijk} denote the event that $F_x(u_{i,k}v_{j,k}) = F_x(v_{j,k}^R u_{i,k}^R)$ for a given i, j, k with $u_{i,k}v_{j,k} \neq v_{j,k}^R u_{i,k}^R$. According to a result from class (the “Alice–Bob” subproblem), $\Pr(E_{ijk}) \leq (\ell - 1)/p$. Thus, the probability that line 17

is executed is (by the union bound) at most

$$\Pr \left(\bigcup_{i=1}^n \bigcup_{j=1}^n \bigcup_{k=0}^{\ell} E_{ijk} \right) \leq \sum_{i=1}^n \sum_{j=1}^n \sum_{k=0}^{\ell} \frac{\ell-1}{p} \leq n^2 \ell^2 / p \leq N^4 / p.$$

Similarly, let E_{ik} denote the event that $F_x(w_{i,k}) = F_x(w_{i,k}^R)$ for a given i, k with $w_{i,k} \neq w_{i,k}^R$. Then $\Pr(E_{ik}) \leq (\ell-1)/p$. Thus, the probability that line 22 is executed is at most

$$\Pr \left(\bigcup_{i=1}^n \bigcup_{k=\ell}^{|s_i|} E_{ik} \right) \leq nN/p \leq N^2/p.$$

By choosing a prime p near N^6 , for example, both probabilities are kept below $1/N^2$. The expected runtime would then be at most $O(N \cdot 1 + N^3 \cdot 1/N^2) = O(N)$.

[*Note:* arithmetic operations are performed only for $\log p = O(\log N)$ -bit numbers.]

[*Remark:* there are other ways to implement line 15. For example, using hashing (or radix sorting), we can get $O(n)$ expected time. (There are more advanced techniques from the string algorithms literature, such as “suffix trees”, that can give $O(n)$ -time deterministic algorithms for this problem. . .)]

[*Remark:* alternatively, the condition that $u_{i,k}v_{j,k}$ is a palindrome can be expressed in another way. Assume $k = |u_{i,k}| \leq |v_{j,k}| = \ell - k$ (the other case is symmetric. . .). We can decompose $v_{j,k}$ into a string $x_{j,k}$ of length $\ell - 2k$ and a $y_{j,k}$ of length k . Then the condition is the same as $u_{i,k}^R = y_{j,k}$ and $x_{j,k}^R = x_{j,k}$. We can again use the fingerprint technique (with slightly more effort to compute the rolling fingerprints of $y_{j,k}$, or by subtracting the fingerprints of two prefixes). . .]